Real-Time Multi-Source Data Integration (Producer–Consumer Pipeline)

1. Methodology

I chose Task 3 because it is fundamentally a systems and data engineering problem rather than a document extraction problem and I wanted to spend the limited time available on building something that actually runs end to end rather than something that looks complete on paper but cannot be demonstrated live.

My approach was to treat each of the five required sources as an independent producer publishing to its own Kafka topic with an explicit schema, then build one Spark Structured Streaming job that subscribes to all five topics, parses each message against its schema, applies the correct temporal tagging rule for that source, resolves and geocodes district names, and writes the harmonized result to partitioned Parquet.

For NIH surveillance data I could not build a reliable PDF table extractor for the real weekly bulletins within the time available alongside everything else, so I generated synthetic but realistic case count data for the same disease categories, districts, and value ranges the real bulletins use. This is disclosed clearly rather than presented as real extraction, and it let me spend the saved time on the parts of the assessment that are unique to Task 3: the Kafka topic design, the Spark consumer, the temporal normalization rule, and the cross source entity linking.

Weather data is pulled live from the Open-Meteo current weather endpoint for each district at producer run time. NDMA disaster alerts, climate policy updates and RSS news items are produced in the formats described in the assessment (event driven, periodic and continuous respectively).

2. Architecture

The diagram below shows the full data flow. Each source has its own producer and topic. A single Spark Structured Streaming application subscribes to all five topics at once, applies harmonization logic per source, and writes partitioned Parquet output. An entity linker then joins the harmonized NIH, weather, and NDMA output on district and epidemiological week to produce a single cross domain dataset, which feeds both the knowledge graph build step and the summary charts.

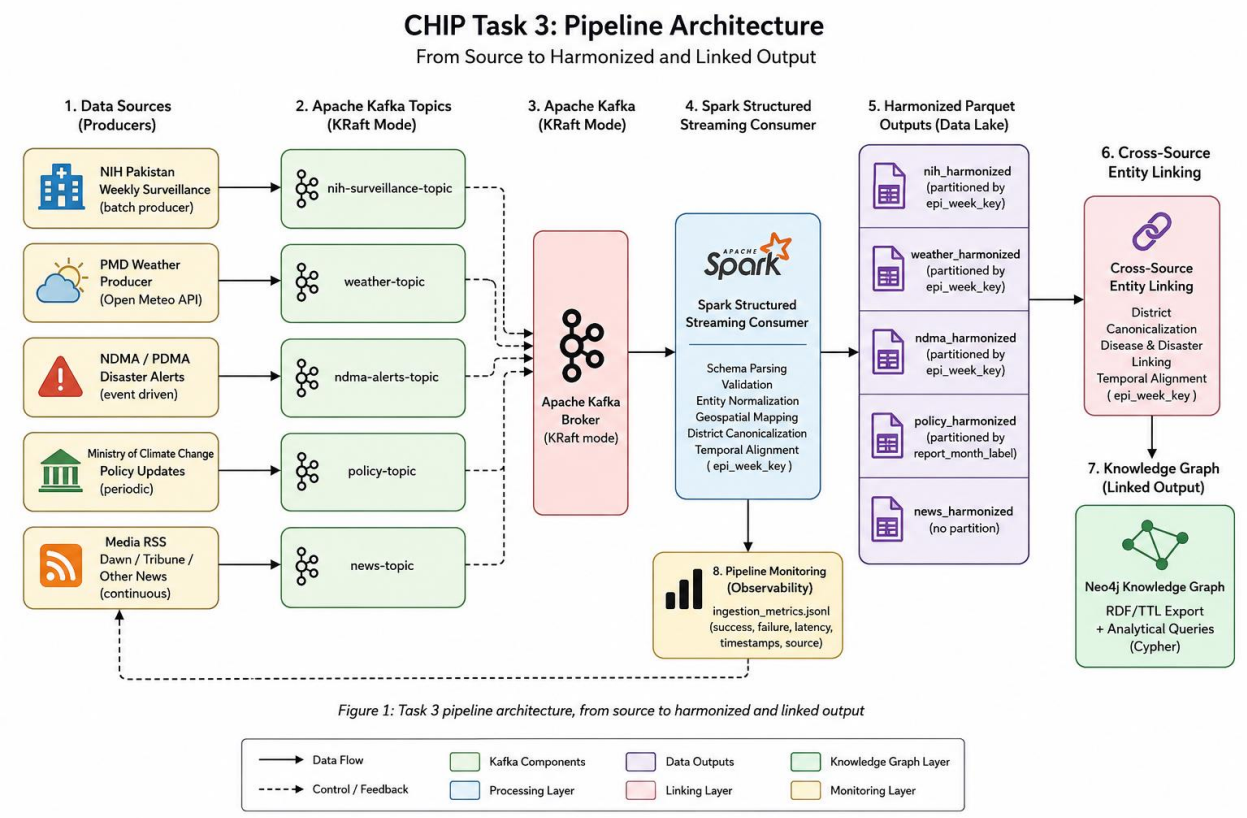

Figure 1: Task 3 pipeline architecture, from source to harmonized and linked output

2.1 Components

- Kafka producers (src/producers): one file per source, each publishing JSON serialized records to a dedicated topic.
- Spark Structured Streaming consumer (src/consumers/spark_consumer.py): subscribes to all five topics, applies a typed schema per topic, tags epidemiological week or report month as appropriate, and writes to partitioned Parquet with a ten second micro batch trigger.
- District resolver and geocoder (src/utils): normalizes district name variants to a canonical name and province, then enriches with latitude and longitude through GeoPy and Nominatim, with a local JSON cache and a settings based fallback.
- Temporal tagger (src/utils/temporal_tagger.py): wraps HeidelTime to extract date expressions from free text fields, such as NDMA alert descriptions, and converts them into the same epidemiological week key used elsewhere in the pipeline.
- Entity linker (src/processors/entity_linker.py): joins harmonized NIH, weather, and NDMA Parquet output on district and epidemiological week, aggregating weather to one row per district per week since it arrives at finer granularity than NIH.
- Monitoring (src/utils/monitoring.py): every producer run logs records attempted, records succeeded, and success rate to a JSONL file.

- Knowledge graph and charts (src/knowledge_graph, src/visualization): an additional layer that builds an RDF graph and summary charts on top of the harmonized output, beyond what Task 3 strictly requires.

3. Pipeline and Schema Design

Each Kafka topic carries a single, explicit schema, defined both at the producer side (Python dictionaries with a fixed set of keys) and the consumer side (a Spark StructType per topic), so a malformed message fails parsing visibly rather than silently corrupting the harmonized output.

Topic	Source	Ingestion mode	Temporal key
nih-surveillance-topic	NIH Pakistan	Batch style (simulated)	Epidemiological week
weather-topic	PMD / Open-Meteo	Near real time API pull	Epidemiological week
ndma-alerts-topic	NDMA / PDMA	Event driven	Epidemiological week (plus HeidelTime extraction from free text)
policy-topic	Ministry of Climate Change	Periodic	Calendar month
news-topic	Dawn, Tribune, The News	Continuous (RSS)	Not applicable, timestamp only

This matches the assessment's temporal normalization requirement directly: NIH, NDMA, and weather records are keyed by epidemiological week, weather is aggregated up to weekly granularity when joined against NIH rather than NIH being collapsed to monthly and only the policy stream, which is inherently periodic, keeps a month level key instead of being forced onto a weekly key.

4. Results

All five producers run successfully and have been exercised multiple times during development. The ingestion log (data/logs/ingestion_metrics.jsonl) records every run with a one hundred percent success rate across NIH, weather, NDMA, policy, and RSS sources in the runs captured so far.

Harmonized output	Rows produced	Partition key
nih_harmonized	56	epi_week_key
weather_harmonized	14	epi_week_key
ndma_harmonized	11	epi_week_key
policy_harmonized	12	report_month_label
news_harmonized	40	none

The cross source linked dataset (outputs/cross_source_linked/nih_weather_ndma_linked.parquet) contains 56 rows, one per NIH disease record, each enriched with the matching district and week's average temperature, humidity, precipitation, and wind speed, plus a count and severity summary of any disaster alerts in that same district and week. For example, several Lahore records for epidemiological week 2026-W27 show a temperature of 31.3 degrees Celsius alongside a concurrent catastrophic severity disaster flag, demonstrating that the join across all three sources works correctly.

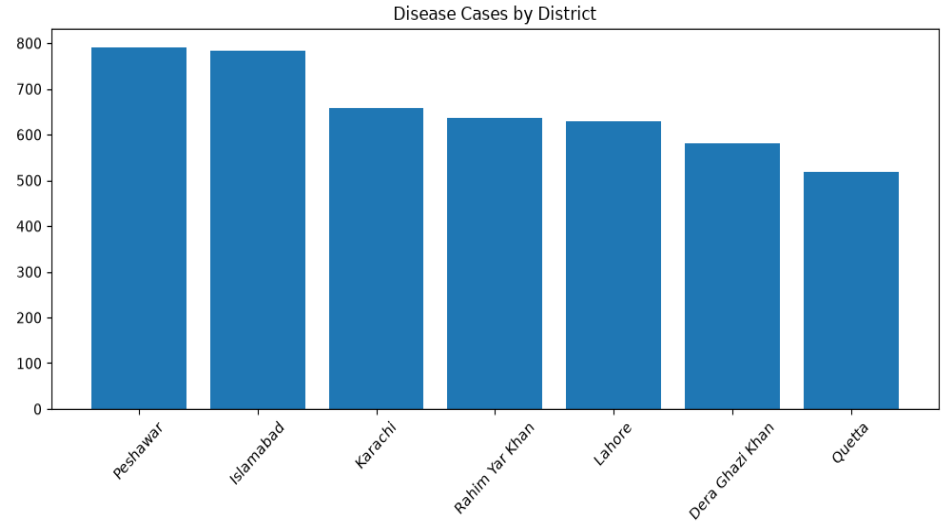

Figure 2: Total simulated disease case counts by district, generated from the harmonized NIH output

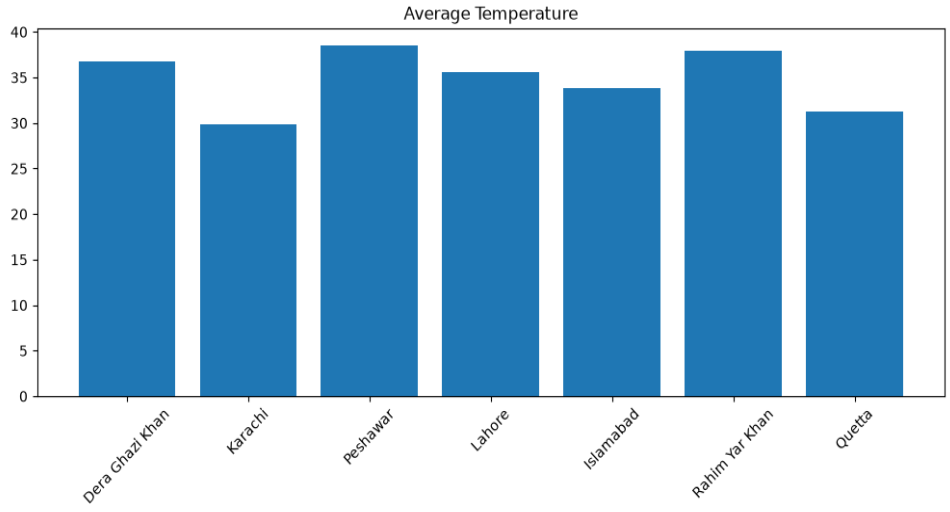

Figure 3: District temperature readings from the harmonized weather output

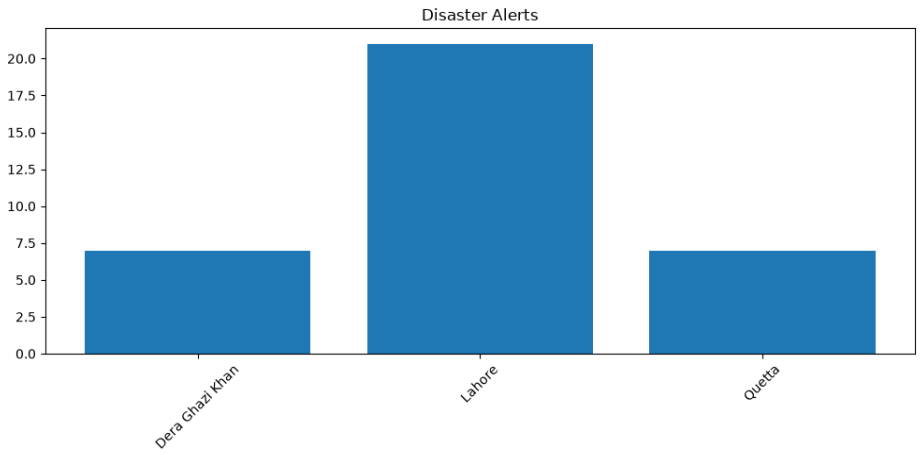

Figure 4: Disaster alert counts by district from the harmonized NDMA output

5. Knowledge Graph (Bonus)

Beyond what Task 3 requires, I loaded the harmonized output into a live Neo4j instance and also opened the RDF export in Protege to inspect the ontology, so the cross domain links could be both queried and visually reviewed. The Neo4j database holds 20 nodes across four labels, Disaster, Disease, District, and Policy, connected by 32 relationships including HAS_DISASTER and HAS_DISEASE. The same four classes appear in Protege's class hierarchy, built directly from outputs/knowledge_graph.rdf. I queried each node type individually in Neo4j to confirm the data loaded correctly across all four categories.

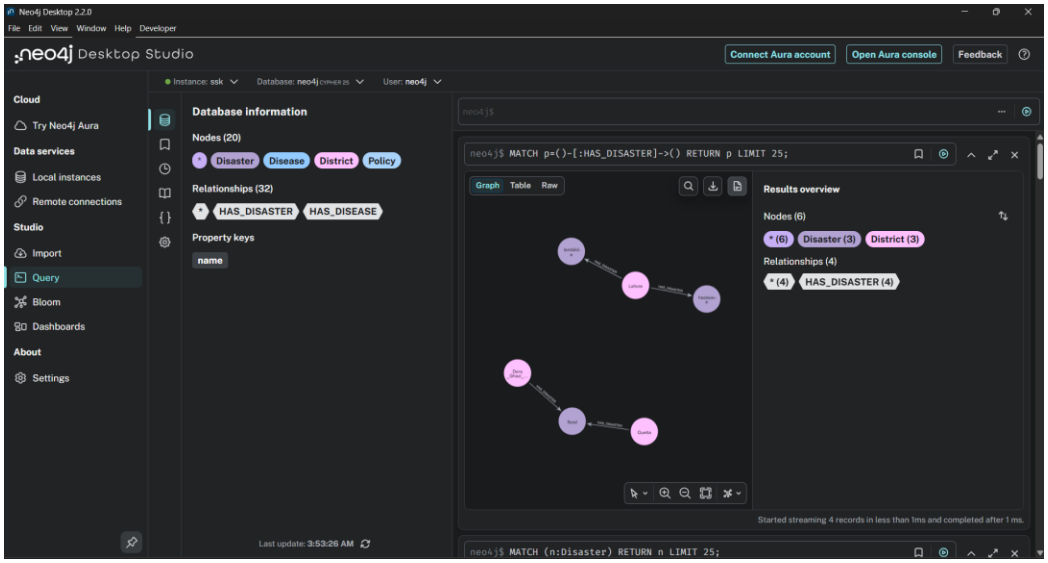

Figure 5: Neo4j Desktop Studio, showing the database information panel and a live Cypher query returning disaster to district relationships

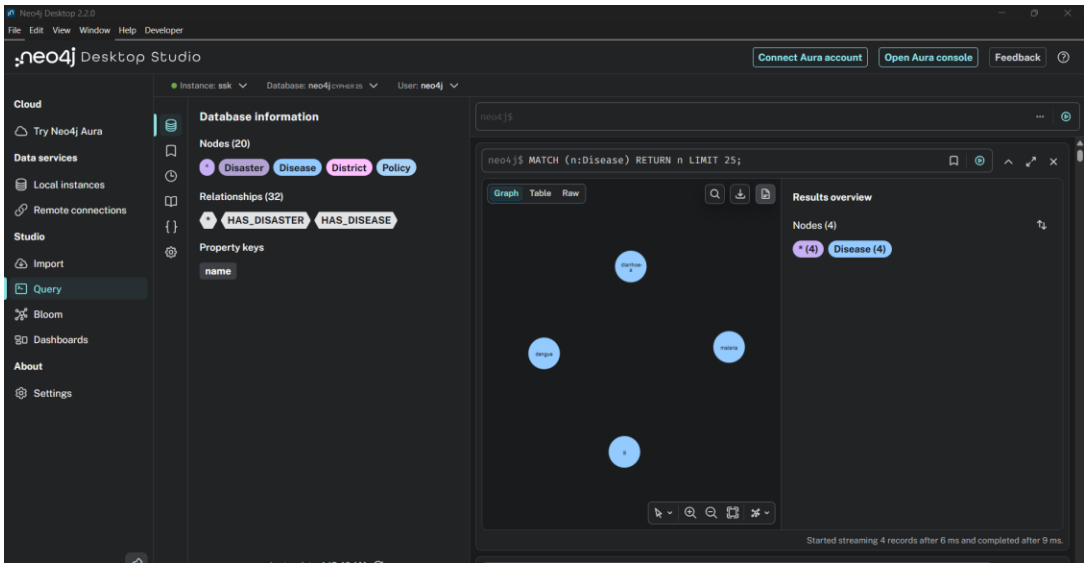

Figure 6: A second Cypher query, MATCH (n:Disease) RETURN n, returning all four disease nodes (dengue, malaria, diarrhoea, ili)

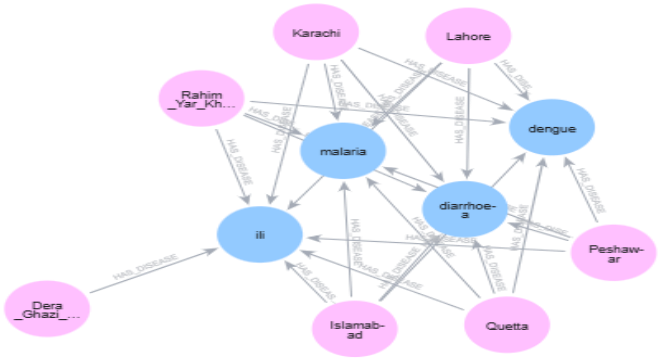

Figure 7: Disease to district relationships explored live in Neo4j, districts in pink connected to disease nodes in blue through HAS_DISEASE relationships

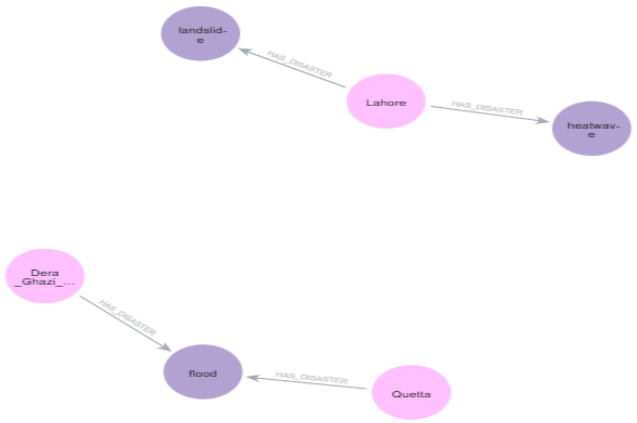

Figure 8: Disaster relationships explored live in Neo4j, districts connected to disaster type nodes (flood, landslide, heatwave) through HAS_DISASTER relationships

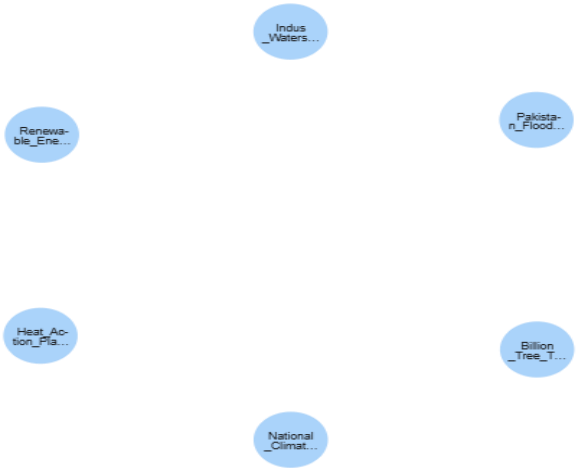

Figure 9: Policy nodes loaded into Neo4j from the Ministry of Climate Change producer output

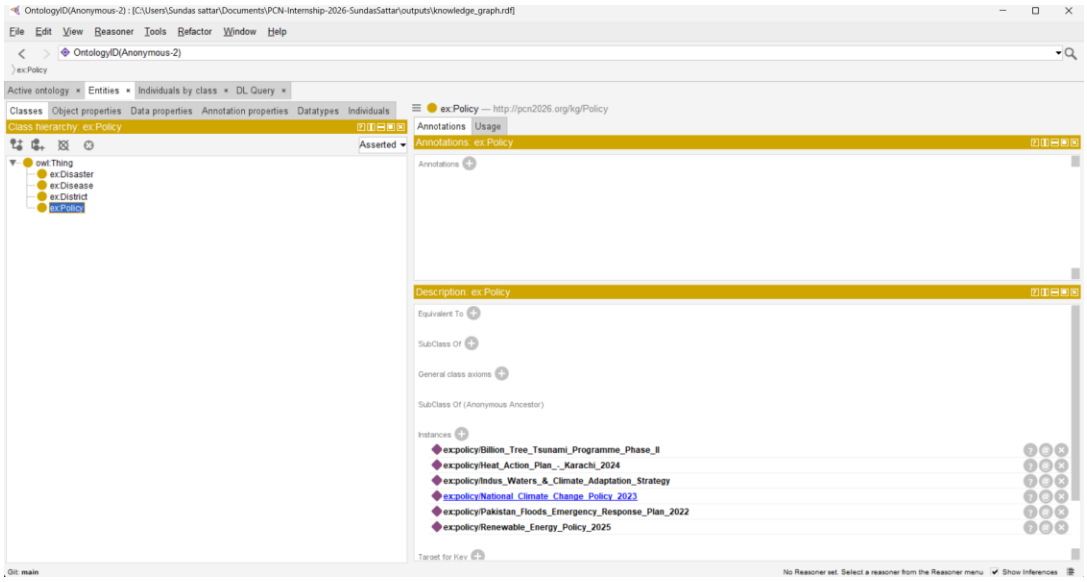

Figure 10: knowledge_graph.rdf opened in Protege, showing the ex:Disaster, ex:Disease, ex:District, and ex:Policy classes, with the six policy individuals listed under ex:Policy

6. Challenges

- Building a real PDF extractor for NIH weekly bulletins on top of the full Task 3 pipeline within 62 hours was not realistic, so I made the call to simulate that one source rather than under deliver on the pipeline engineering itself and documented that choice instead of hiding it.
- Getting Spark Structured Streaming's checkpoint and offset handling right took a few iterations, particularly making sure each topic's stream restarts cleanly from its own checkpoint directory without interfering with the others.
- Nominatim has a strict request rate limit, so the geocoder needed a local cache and a settings based fallback to avoid the pipeline stalling or failing when geocoding the same district repeatedly across producer runs.
- Deciding the exact join key for the entity linker took some thought, since NIH, weather, and NDMA data arrive at different natural granularities. Settling on district plus epidemiological week, with weather pre aggregated to that grain before the join, was the cleanest way to satisfy the assessment's instruction not to force everything onto NIH's or PMD's native resolution.

7. Use of LLMs

I used ChatGPT and Claude throughout this project and I want to be specific about where they helped and where the work was mine.

Before picking a task, I used both tools to walk through all three options so I understood what each one actually required. Both suggested Task 1 as the safer, less time pressured choice. I went a different way and chose Task 3, the real time pipeline, because it matched what I wanted to build and test as a developer, not because an LLM told me to.

Once I had committed to Task 3, the design decisions were mine. I decided to split the work into five independent producers, one per source, each with its own Kafka topic and schema. I decided that

surveillance and disaster data should be keyed by epidemiological week while policy data should stay at month granularity, since that is what the source data actually supports. I decided how district name variants should be resolved to a canonical form, and how the entity linker should join NIH, weather, and NDMA data together on district and week. I evaluated both Protégé and Neo4j for knowledge graph visualization. The final solution exports the RDF knowledge graph, imports it into Neo4j for interactive exploration using Cypher queries, and also provides RDF visualization in Protégé.

With that design in hand, I used Claude and ChatGPT to help implement it in code, for example generating a first version of a Kafka producer for a given schema, or a Spark Structured Streaming consumer that applies a specific partitioning rule. I did not treat their output as final. LLMs are fast but they hallucinate and miss details, so I checked what they gave me against what I had actually designed, sent corrections back when something was wrong or missing, and tested every piece myself by running the producers and the consumer and checking the output. The debugging, in particular, was a back and forth process: when something failed, I described the error, worked through what it meant, and used the tools to help me fix it faster than I could have alone given the time constraint.

So in short, the architecture, the decisions and the verification are mine. The first draft of most of the implementation came from Claude and ChatGPT, directed by my design and corrected through my testing. Additionally, the architecture diagram was generated with the assistance of ChatGPT based on my implemented system architecture and was reviewed to ensure it accurately represents the final implementation.

I also used Claude to help draft and organize the wording of this report and the README. All the results, metrics, and visualizations included, the knowledge graph output, the charts, and the figures, are real outputs from my own pipeline runs, not generated or invented by the AI. Claude helped me put them into a clear written structure based on what I gave it, and I reviewed the wording throughout to make sure it matched what I actually built and did.

8. Declaration

I certify that this submission is my own work.

I have disclosed all use of ChatGPT and Claude above. I did not use Gemini or Copilot for this submission.

I understand that plagiarism may result in disqualification.

Name: Sundas Sattar

Date: 1-July-2026